PennState College of Engineering

HAROLD AND INGE MARCUS DEPARTMENT OF INDUSTRIAL AND MANUFACTURING ENGINEERING

Research

NeurMHSP

Taehyeon Kwon

Table of contents

1 Overview

2 Today work

3 Future Work

Overview

Overview

Completed

- Implemented data generation code, adhering to the rules denoted in the CPLP paper.
- Implemented the Q function to compute the expected second-stage value.
- Set up Gurobi.
- Saved and loaded the network model using ONNX.

In Progress

- Fixing errors related to infeasibility occurrences during second-stage value calculations.
- Gaining GPU access.

Today work

Capacitated Plant Location Problem (CPLP)

Obejctive: Minimize the total cost, which is the sum of:

- Fixed costs associated with opening each facility.
- Variable costs for serving customer demands from opened facilities.

Mathematically, it can be expressed as:

$$\min Z = \sum_j f_j y_j + \sum_i \sum_j t_{ij} x_{ij}$$

where f_j and t_{ij} represent fixed and transportation costs, respectively, and y_j and x_{ij} are decision variables.

We can formulate as follows:

$$
\begin{aligned}
\min_{\mathbf{x},\mathbf{y}} \quad & \sum_j f_j y_j + \sum_i \sum_j t_{ij} x_{ij} \\
\text{s.t} \quad & y \ \text{ interger}, \quad y \in \{0, 1\} \\
& \sum_j x_{ij} = 1 \quad \forall_i \\
& \sum_i d_i x_{ij} \leq c_j y_j \quad \forall_j \\
& 0 \leq x_{ij} \leq 1 \quad \forall_{i,j}
\end{aligned}
$$

- f_j: fixed cost of plant j
- t_{ij}: transportation cost of client i met from plant j.
- d_i: demand of client i
- c_j: capacity of plant j

What we do?

We modified the CPLP problem as two stage stochastic programming manner.
It can be expressed as:

$$\min_{\mathbf{y},\mathbf{x}} Z = \sum_j f_j y_j + \hat{Q}(y_j)$$

$$\text{s.t. } \; y \; \text{ interger}, \;\; y \in \{0, 1\}$$

$$\hat{Q}(y_j) = \frac{1}{N} \sum_{i=1}^{N} Q(y_j, \xi_i) \qquad (let, \xi_i = \{d_i\})$$

, where $\hat{Q}(y_j)$ denotes the expected second stage value. The function returns the expected second stage value by calculating the average of second stage value across all scenarios, given a fixed first-stage decision (y)

The function $Q(\cdot)$ represents the optimization problem for the second stage of the CPLP problem. It returns the optimal second-stage value for each scenario, given a first-stage decision y.

$$
\begin{aligned}
Q(y_j, \xi_i) = \min_{\mathbf{x}} \quad & \sum_i \sum_j t_{ij} x_{ij} \\
\text{s.t} \quad & \sum_j x_{ij} = 1 \quad \forall_i \\
& \sum_i d_i x_{ij} \leq c_j y_j \quad \forall_j \\
& 0 \leq x_{ij} \leq 1 \quad \forall_{i,j}
\end{aligned}
$$

Data Generation

Cornuejols et al. (1991) used techniques to generate parameters, namely t, d, c and f:

- **Transportation cost** (t): $10\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}, \quad x_{\{a,b\}}, y_{\{a,b\}} \sim U[0, 1]$
  - $U[a, b]$ denotes the uniform distribution in the interval $[a, b)$.
- **Demand** (d): $U[5, 35]$
- **Capacity** (c): $U[10, 160]$
- **Fixed cost** (f): $U[0, 90] + U[100, 110]\sqrt{c_j}$

We made 3 sets of 300 problems. In each set, there are 100 problems of each of the following sizes: 10×10, 25×25, 50×50 , where the first number is the number of clients and the second is the number of plants.

Architecture

Three Python files are utilized for implementation:

- `data_generation.py`: Responsible for data generation.
- `CPLP.py`: Defines the CPLP model and solver, and collects results.
- `func_Q.py`: Calculates the expected second-stage value with a given first-stage decision from `CPLP.py`.

Two types of data files are created:

- `scenario_{i}.dat`: A data file containing parameters for scenario i, where $i \in \{0, \ldots, 99\}$.
  - These are stored in individual directories by problem size: `'data/CPLP_{# of clients}_{# of plants}/'`.
- `result_{# of clients}_{# of plants}.csv`: A data file containing the first-stage decision and expected second-stage value for each scenario of each problem size.

Graphically,

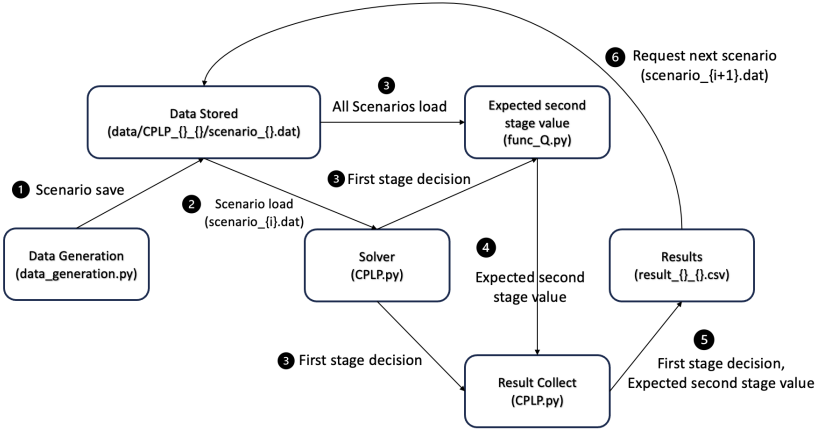

Figure: Implementation Architecture

Conclusion

- Completed building the architecture and verified its functionality.
- Encountered infeasibility issues during the optimization of the second-stage decision in `fun_Q.py`.
  - Implemented the Big M method to derive a solution.
  - A clear formulation with the Big M method is essential.
  - I repeatedly tried to formulate and solve the problem with the Big M method, but failed.
- Upon finalizing the formulation, datasets will be created to train the neural network(NN) model.
  - Codes for training NN model are ready.
  - We can save and load trained NN model using onnx.

Future Work

Future works

- Continue to develop the architecture and logic.
- Compare the performance of the Neural2SP model with our model, which was developed without permutation inference.
- Create a document for the deterministic EMPIRE model using LaTeX.

PennState College of Engineering
HAROLD AND INGE MARCUS DEPARTMENT OF INDUSTRIAL AND MANUFACTURING ENGINEERING

Taehyeon Kwon

Research

NeurMHSP